

Core Supervised Learning

Preprocessing, Models and Evaluation

Preprocessing choices

The hold-out test set from the Day 1 baseline notebook was reproduced exactly (same cleaning steps, same `test_size=0.20` stratified split, same `random_state=42`), which was confirmed by recomputing the rule baseline's precision on this notebook's test set and getting the same 0.475 Day 1 reported.

`fnlwgt` (a census sampling weight with no business meaning for an individual) was dropped from the feature list before building the pipeline. `native-country` was grouped into US vs Non-US before the split, a fixed rule rather than a learned statistic, so it introduces no leakage.

The `ColumnTransformer` uses median imputation and `StandardScaler` on the numeric block, and a constant "Missing" imputer with `OneHotEncoder(handle_unknown='ignore')` on the categorical block. Median was chosen over mean because `capital-gain` and `capital-loss` are heavily skewed, so the mean would be pulled toward a handful of large outliers. A constant "Missing" category was chosen over most-frequent imputation because filling missing `workclass` or `occupation` values with the most common category would misrepresent people whose employment status was genuinely not recorded, and Day 1's error analysis suggested missingness itself might carry signal. One-hot encoding was preferred over ordinal encoding because none of the categorical columns (`workclass`, `occupation`, `marital-status`, and so on) have a natural order; assigning them arbitrary integers would invent false distances between categories. Target encoding was considered for high-cardinality columns like `occupation` but skipped today because it uses the target during preprocessing and is easy to leak if not handled carefully inside cross-validation, which adds complexity better suited to a later iteration.

Model comparison:

Model	Accuracy	Precision	Recall	F1	ROC AUC	PRAUC
Majority class (Day 1)	0.761	0.000	0.000	0.000	—	—
Rule: <code>edu>=13</code> OR <code>capgain>0</code> (Day 1)	0.745	0.475	0.603	0.531	0.697	0.381
Decision Tree (unrestricted)	0.829	0.647	0.629	0.638	0.781	0.522
Logistic Regression	0.851	0.730	0.601	0.659	0.905	0.765

Both real models clear both Day 1 baselines by a wide margin. Logistic Regression beats the Decision Tree on every metric except recall, where the two are close (0.601 vs 0.629). Its ROC and precisionrecall curves sit above the tree's across nearly the entire threshold range, not just at the default 0.5 cutoff.

Error profile and the business metric.

Both models produce more false negatives than false positives, which works in favor of the precision-focused business goal set in Day 1 (a false positive is a wasted contact, a false negative is only a missed opportunity). Logistic Regression leans into this harder: 520 false positives vs the Decision Tree's 803, which is the direct driver of its higher precision.

Overfitting check.

The unrestricted Decision Tree reached depth 49 with 6,032 leaves, scoring 97.7% accuracy on the training set but only 82.9% on the test set, a 15-point gap that is a clear overfitting signature. Its toplevel

splits (marital-status_Married-civ-spouse, then capital-gain, then education-num) are sensible and echo the strongest Logistic Regression coefficients, but by 40+ levels deep the tree has almost certainly stopped learning general patterns.

Which model to develop further.

I'm going with Logistic Regression. It comes out ahead of the Decision Tree on almost every metric (precision 0.730 vs 0.647, recall 0.601 vs 0.629 is the only close one, F1 0.659 vs 0.638, ROC AUC 0.905 vs 0.781, PR AUC 0.765 vs 0.522), and its ROC and PR curves stay above the tree's across most threshold values, not just at the default cutoff. Its coefficients are also easy to explain. Capital gain, marital status, education, and occupation all show up with signs that make sense, which matters if a business stakeholder ever asks why the model flagged someone.

I'm not dropping the Decision Tree completely though. The version I built today was never meant to be final, it was unrestricted on purpose so I could see how badly it overfits. A depth-limited or leaf-limited version is worth trying tomorrow for two reasons: trees can pick up on non-linear patterns (like the capital-gain thresholds that showed up in the top splits) that a linear model just can't represent, and a shallow tree might actually be easier to explain to a non-technical person since it reads like a flowchart of questions. If a pruned tree still falls well behind Logistic Regression after tuning, I'll stick with Logistic Regression as my main model for the rest of the week.

Preprocessing changes to test tomorrow.

Three things came up today that are worth fixing next: limiting the Decision Tree's max_depth and min_samples_leaf so it stops overfitting, log-transforming or bucketing capital-gain and capital-loss since they're mostly zero with a long tail, and taking a closer look at how occupation handles categories the model hasn't seen before, right now those just get zeroed out instead of dealt with properly.